🧭 The xaiht.org Git Survival Guide

Exactly what to type **every time you ENTER** and **every time you LEAVE** the project.

For **Angela López Mendoza** · July 9, 2026 · Repo: `C:\Development\XAIHT\web-app` · Keep this next to the keyboard.

> **Read this first.** You are **not** stupid. The July 9 failure was a silent tool detaching HEAD at 4 AM — the investigation cleared you completely. Git's messages are cryptic for *everyone*; that's git's fault, not yours. This guide exists so you never have to memorize anything. And the universal escape hatch, valid in every scenario on this page and every scenario NOT on this page: **copy the exact message git printed and paste it to Claude.** That is always a correct move. Nothing here can kill your work — git refuses before it destroys.

⚡ The One-Page Cheat Sheet

What you see	What you do
You just sat down to work	`git status` — always the first command, every time
`On branch main` + `working tree clean`	`git pull --ff-only` → start working ✅
Red *modified / untracked* files	`git add -A` → `git commit -m "message"`
`HEAD detached at ...`	`git checkout main` → `git status` again
checkout warns: "*leaving N commits behind:* ***abc1234***"	`git merge --ff-only abc1234` → `git push`
`On branch` *anything-not-main*	`git checkout main` → tell Claude the branch name
"*Your branch is* ***ahead*** *of 'origin/main'*"	You forgot to push last time → `git push`
"*Your branch is* ***behind*** *'origin/main'*"	`git pull --ff-only`
pull says "*Not possible to fast-forward*"	🛑 **STOP.** Type nothing else. Call Claude.
push says "*Everything up-to-date*" **right after a fresh commit**	🚨 ALARM → `git status`; if detached → checkout main; else call Claude
You're leaving	commit → `git push` → `git status` must show clean + up to date
You want **the website** to show your changes	Docker Desktop → Jenkins `localhost:8080` → **xaiht-deploy** → **Build Now** → green → Ctrl+F5 on xaiht.org
Anything weird, any refusal, any red text you don't recognize	STOP. Copy the exact message. Paste to Claude. **Never** add `--force` to anything.

🌅 PART 1 — ENTERING (every single time, ~20 seconds)

Step 1. Open the terminal in `C:\Development\XAIHT\web-app` (the VS Code terminal is fine).
Step 2. Type:

```
git status
```

Step 3. Read what it printed and find your scenario below. There are only seven.

Scenario A — Good morning NORMAL

It says `On branch main` and `nothing to commit, working tree clean`.

```
git pull --ff-only
```

- "*Already up to date.*" → perfect, start working.
- "*Fast-forward*" + a list of files → newer versions came down from GitHub. Start working.
- Anything with "*fatal*" or "*Not possible to fast-forward*" → go to **Scenario G**.

Scenario B — Leftover red files from last time NORMAL

`On branch main`, but there are red files under "*Changes not staged*" or "*Untracked files*". That's unsaved-to-git work from your last session. The rule: **when in doubt, COMMIT.** A commit never destroys anything, and junk can always be removed later with a *new* commit.

```
git add -A
git commit -m "leftover work from last session"
git pull --ff-only
```

If you suspect the red files are garbage: do **not** delete blindly — look first with `git diff`, or ask Claude.

Scenario C — `HEAD detached at ...` THE JULY-9 GREMLIN

Not your fault — a tool does this. Breathe. Type:

```
git checkout main
```

Then read what it printed:

- **Nothing scary** → `git status` → continue at Scenario A or B. Done.
- **"Warning: you are leaving N commits behind: `abc1234` ..."** → those commits must be rescued *right now* (this is exactly what stranded v1.38.1). Use the sha it printed:

  ```
  git merge --ff-only abc1234
  git push
  git log --oneline -3   ← the rescued commits should be at the top
  ```

- **It refuses:** "*your local changes would be overwritten by checkout*" →

  ```
  git stash
  git checkout main
  git stash pop
  ```

  then do Scenario B (commit). If any of these three errors → STOP, call Claude.

Scenario D — On a branch that isn't `main` CLEANUP NEEDED

You only ever work on `main`. Type `git checkout main`, then tell Claude the branch's name so it gets cleaned up properly.

Scenario E — "*Your branch is ahead of 'origin/main'*" EASY

You committed last time but forgot to push. Just: `git push`

Scenario F — "*Your branch is behind 'origin/main'*" EASY

GitHub has newer commits than your PC. Just: `git pull --ff-only`

> **Scenario G — pull refuses: "*Not possible to fast-forward*"** STOP
>
> Your PC and GitHub disagree in a way that needs eyes. **Do nothing else.** No plain `git pull`, no `--force`, no clicking things in a GUI. Copy the full message and give it to Claude. This is rare and always fixable — forward-only.

🔨 PART 2 — WHILE WORKING

- Save files in the editor, then commit **small and often**:

  ```
  git add -A
  git commit -m "what I just did"
  ```

  Commits are free save-points. There is no such thing as too many.
- If a commit is **BLOCKED** by the big **DETACHED HEAD** warning — the guard just *saved* you. Do Scenario C, then commit again.
- Know the three levels: **commit** = save on your PC · **push** = upload to GitHub · **Jenkins Build Now** = update the real website. Each one does NOT do the next one automatically.

🌙 PART 3 — LEAVING (every time, ~60 seconds)

Step 1 — Commit everything

```
git status
```

Red files? →

```
git add -A
git commit -m "describe what you did today"
```

Step 2 — Push

```
git push
```

✅ **GOOD:** output ends with a line like `main -> main`. Your work is on GitHub.

🚨 **ALARM:** it says "*Everything up-to-date*" **immediately after you just committed**. That means your commit did NOT reach GitHub (this was the July-9 symptom). Run `git status`: if it says detached → Scenario C. Otherwise → call Claude. Do not shrug this off.

Step 3 — The safe-to-walk-away check

```
git status
```

You may close the laptop only when you see **all three**:

1. `On branch main`

2. `Your branch is up to date with 'origin/main'`
3. `nothing to commit, working tree clean`

Step 4 — OPTIONAL: publish to the real website

Only when you want **https://xaiht.org** to show the new version:

1. Start **Docker Desktop** and wait until its containers are running.
2. Browser → `http://localhost:8080/job/xaiht-deploy/` (your local Jenkins login).
3. Click **Build Now**. Wait for the build to turn **green**.
4. Open **https://xaiht.org** and hard-refresh: **Ctrl+F5**.

- Jenkins page won't load → Docker Desktop isn't up yet. Start it, wait, retry.
- Build turns **red** → click the build number → *Console Output* → copy the last ~30 lines to Claude.
- Site still looks old after a green build → Ctrl+F5 again, or try a private window (it's browser cache).

> ⚠️ **Remember:** push ≠ deploy. If you skip Build Now, the site stays on the old version — that is *normal*, not broken.

📜 The Golden Rules

1. `git status` **FIRST** — entering and leaving. It's your headlights.
2. **When in doubt, COMMIT.** Commits never hurt. Junk is removed with a *new* commit, never by rewriting.
3. Pull **only** as `git pull --ff-only`. Never plain `git pull`.
4. If git **refuses**, it is *protecting* you. Stop, read, or call Claude. **NEVER** add `--force` to anything.
5. **Never rewrite history:** no rebase, no `--amend`, no `reset --hard`, no force-push. Ever.
6. "*Everything up-to-date*" right after you committed = **alarm**, not reassurance.
7. Push uploads code; only Jenkins **Build Now** updates the website.
8. Anything weird → copy the exact message → paste it to Claude. Always a correct move.

Written for Angela López Mendoza by Claude · July 9, 2026 · The detached-HEAD guard is installed at `.git/hooks/pre-commit` and will loudly block off-branch commits (if you ever re-clone the repo, ask Claude to reinstall it). You recovered everything today — this guide makes sure next time takes 30 seconds, not an hour.